

# Practica 10 - Paradigmas de Lenguaje de Programación

1. Un programa funcional es una gran expresión matemática compuesta por funciones puras. Es una colección de definiciones de funciones. No hay sentencia de asignación. El rol de la computadora es el de una calculadora matemática gigante.
2. El lugar donde se definen las funciones se conoce como Script (archivo de texto de código fuente), el cual al procesarse genera un Entorno de definiciones inmutables que la computadora utilizará para evaluar expresiones.
3. El concepto de variables en los lenguajes funcionales es completamente distinto al de los lenguajes tradicionales como C, Java o Python. Una variable en un lenguaje funcional es una constante desde el punto de vista imperativo. No representan celdas de memoria que mutan, sino nombres fijos asignados a valores fijos.
4. Una expresión en programación funcional es una fórmula lógica o matemática. Su valor depende de los datos que recibe, garantizando que una misma expresión, con los mismos datos, devuelva el mismo resultado.
5. La forma de evaluación de los lenguajes funcionales es través de mecanismo de reducción o simplificación. Se le llama reducción porque la expresión original suele ser una estructura grande, anidada y compleja (función dentro de otra, que a su vez está dentro de otra). El trabajo de la computadora es ir desarmando esa estructura resolviendo las partes internas o externas hasta que la montaña de funciones se “reduce” a un dato: número, texto o una lista.
6. Los lenguajes funcionales son fuertemente tipados porque necesitan consistencia matemática estricta. Al no haber memoria que cambie, ni instrucciones paso a paso, los tipos de datos son las vías que aseguran que la reducción y simplificación de las expresiones se haga de forma correcta y sin accidentes.
7. Un programa escrito en POO es un conjunto de objetos que interactúan mandándose mensajes. Cada objeto tiene estado (variables) y comportamiento (métodos). La ejecución del programa ocurre cuando un objeto le pide a otro que haga algo. Un mensaje es la invocación de un método.
8. Los elementos más importantes en la POO son:
  - Clase: molde o plantilla a partir del cual se crean los objetos. La clase define que datos va a tener ese tipo de objeto (atributos) y que acciones va a poder realizar (métodos).

- Objeto e Instancia de Clase: un objeto es la entidad real y concreta que se crea a partir de una clase. El proceso de crear un objeto usando ese molde (objeto) se llama instanciación. Por eso, decir “objeto” o “instancia de una clase” es prácticamente hablar de lo mismo.
- Métodos: funciones o acciones que un objeto sabe realizar. Están definidos dentro de la clase y determinan el comportamiento del objeto.
- Mensajes: forma en que los objetos se comunican e interactúan entre sí. Es una petición que un objeto le envía a otro para que ejecute uno de sus métodos.
- Herencia: mecanismo que permite crear una clase nueva a partir de una clase existente, heredando sus atributos y métodos. Sirve para reutilizar código y organizar información jerárquicamente.
- Polimorfismo: es la capacidad que tienen objetos de distintas clases de responder al mismo mensaje de maneras diferentes.
- Encapsulamiento: ocultar los detalles internos de cómo está hecho y mostrar solo lo necesario a través de sus métodos. Nadie puede tocar los datos de un objeto directamente si este no lo permite.

9. El segundo nivel de abstracción en la POO es la Herencia. La jerarquía de niveles funciona así:

- Primer nivel (Encapsulamiento): te abstraes de los detalles internos de un solo objeto. Agrupas datos y comportamientos en una capsula y ocultas como funciona por dentro.
- Segundo nivel (Herencia): te abstraes de las similitudes entre múltiples clases. Ya no miras un objeto aislado, sino que miras el panorama completo y empiezas a encontrar patrones, clasificaciones y jerarquías en tu sistema

La abstracción consiste en quitar detalles específicos para quedarte con el concepto general. La herencia hace:

1. Encontrar el denominador común: tienes un sistema para una veterinaria con las clases Perro, Gato y Loro. En el primer nivel, encapsulas cada uno por separado. Pero en el segundo nivel, te das cuenta de que todos comparten cosas: nombre, edad y emiten un sonido.
2. Creas una abstracción: creas una clase abstracta llamada Animal (superclase o clase padre)
3. Definís las reglas generales en Animal y dejas que Perro o Gato hereden todo eso preocupando solo por lo que los hace únicos.

10. Las formas en que las clases pueden heredar de otras, se clasifican según cuántos padres puede tener una clase y cómo se organiza esa estructura:

- Herencia Simple: una clase hija hereda de una sola clase padre.

- Herencia Múltiple: una clase hija puede heredar atributos y métodos de dos o mas clases padre al mismo tiempo.
- Herencia Jerárquica: una sola clase padre sirve como base para muchas clases hijas
- Herencia Multinivel (o en cascada): cadena de herencia. Una clase hereda de otra clase hija, convirtiéndose en “nieta”.

Smalltalk utiliza estrictamente Herencia Simple, una clase solo puede tener un padre directo. Como solucionar la necesidad de herencia múltiple: se suele usar la composición (un objeto que contiene a otros) o mecanismos como los Traits (módulos de métodos que se pueden pegar a cualquier clase sin usar herencia).

C++ soporta Herencia Múltiple (además de la simple). Permite declarar que una clase herede de tantas clases como se quiera. Esto introduce el problema de lógica llamado El Problema del Diamante: tenemos una clase Vehiculo con el método arrancar(). Las clases Auto y Barco heredan de Vehiculo. Si creas la clase AutoAnfibio heredando de Auto y Barco y llamas al método arrancar(), cual de los dos métodos debería ejecutar? el de Auto o el de Barco? esto genera una ambigüedad que C++ obliga al programador resolver usando cosas complejas como herencia virtual.

11. En el paradigma lógico, las variables son elementos indeterminados (incógnita) que pueden sustituirse por otro, el motor lógico va a rellenar con constantes. Y las constantes son elementos que ya estan definidos (concreto y fijo) y no se pueden cambiar por otra cosa.
12. Escribir un programa en un lenguaje lógico consiste en crear un archivo de texto con hechos y reglas que describen un problema, para luego hacerle consultas en una consola y dejar que el motor de inferencia de la computadora deduzca las respuestas automáticamente. Por ejemplo:
  - a. Le enseñamos a la computadora que series existen, de que genero son y que le gusta a cada usuario:

```
genero(dark, ciencia_ficcion)
genero(the_office, comedia)
le_gusta(juan, ciencia_ficcion)
le_gusta(maria, comedia)
```

- b. Creamos una regla para que el sistema aprenda a recomendar una serie. A un usuario se le recomienda una serie si a ese usuario le gusta un genero y ademas esa serie es de ese mismo genero:

```
recomienda(Usuario, Serie) :- le_gusta(Usuario, Genero), genero(Serie, Genero)
```

- c. Hacer las consultas:

?- recomienda(juan, dark) repuestos de prolog: true  
?- recomienda(juan, friends) respuesta de prolog: false

13.

- Imperativo en Python:

```
num = int(input("Ingrese un número: "))
if numero % 2 == 0:
    print("El valor ingresado es PAR")
else:
    print("El valor ingresado es IMPAR")
```

- Declarativo:

- Lógico en Prolog:

```
evaluar(X) :- 0 is X mod 2, write('El valor ingresado es PAR'), nl.
evaluar(X) :- 1 is X mod 2, write('El valor ingresado es IMPAR'), nl.
comenzar :-
    write('Ingrese un numero: '),
    read(Num),
    evaluar(Num).
```

// para ejecutarlo se hace la consulta → ?- comenzar.

- Funcional en Haskell:

```
evaluarParidad :: Int -> String
evaluarParidad x = if even x then "El valor ingresado es PAR" else "El valor es IMPAR"
```

```
main :: IO ()
main = do
    putStrLn "Ingrese un numero:"
    entrada <- getLine
    let num = read entrada :: Int
    putStrLn (evaluarParidad num)
```

- Orientado a Objetos en Java:

```
import java.util.Scanner;

public class Verificador {
    public void chequearParidad(int numero) {
        if (numero % 2 == 0) {
```

```

        System.out.println("El valor ingresado es PAR");
    } else {
        System.out.println("El valor ingresado es IMPAR");
    }
}

public static void main (String[]args) {
    Scanner teclado = new Scanner(System.in);
    System.out.print("Ingrese un numero:");
    int num = teclado.nextInt();
    Verificador verificador = new Verificador();
    verificador.chequearParidad(num);
}
}

```

14. Los Lenguajes Basados en Scripts nacieron para automatizar tareas repetitivas y conectar programas entre si. Se utilizan para desarrollo web, IA, administración de sistemas, etc. Características:

- No requieren proceso de compilación para generar un archivo binario(.exe). El código fuente se lee y ejecuta mediante un programa llamado interprete
- Diseñados para escribir código rápido.
- Cuentan con un recolector de basura (Garbage Collector)
- Excelentes para interactuar con SO, ejecutar comandos de consola, leer/escribir archivos y comunicar programas escritos en diferentes tecnologías.

En el servidor: Python, Ruby y PHP

En el cliente: JS

En SO: Bash/Shell, PowerShell

Utilizan tipificación de tipado dinámico: no se declara el tipo de dato, una variable adquiere el tipo automáticamente en tiempo de ejecución según el valor que se le asigne y puede cambiar más adelante como en Python. Y también utilizan tipado débil con excepciones, como JS o PHP, realizan conversiones automáticas implícitas de tipos, cuando se intenta operar con tipos de datos diferentes, intentan adivinar.

15. Si, existen muchos otros paradigmas. Un solo paradigma no puede resolver todos los problemas del software de manera eficiente. A medida que la tecnología avanza y surgen nuevas necesidades, los programadores necesitan nuevas formas de estructurar su pensamiento y su código:

- Paradigma Dirigido por Eventos: el programa queda escuchando, cuando ocurre un suceso, como hacer clic, el sistema dispara una función específica para manejar ese evento. Se usa en interfaces gráficas, desarrollo de videojuegos y aplicaciones web con JS
- Paradigma Orientado a Aspectos: apunta a dividir el programa en módulos independientes c/ uno con un comportamiento y responsabilidad. Escribir una

sola vez en un aspecto en vez de escribir el código de seguridad en 50 clases distintas. Se usan en frameworks

- Programación Reactiva: se basa en reaccionar a flujos de datos asíncronos. El programa se suscribe a un flujo y reacciona de forma automática cada vez que un dato nuevo aparece manteniendo la app rápida y sin trabarse.